Quantum Machine Learning

From classical learning to a quantum convolutional neural network — explained step by step

CUDA-Q Quantum Algorithm Benchmarking Suite

Companion lecture to `QML.ipynb`

All quantum results in these slides were computed with NVIDIA CUDA-Q 0.16.0 (`qpp-cpu`, no GPU) using the kernel of `QML.ipynb`, and checked against an independent NumPy simulation.

What you will learn today

A. **Classical machine learning**: data, models, losses, gradient descent, neural networks, evaluation

B. **Quantum computing** for QML: qubits, gates, entanglement, measurement, parameterized circuits

C. **QML fundamentals**: data encoding, feature maps, kernels, variational circuits, hybrid training, parameter shift

D. **QML algorithms**: VQC, QSVM / kernels, QNN, QCNN, QGAN, qPCA, clustering, quantum RL

E. **Advanced topics and implementation**: barren plateaus, noise, the CUDA-Q code of `QML.ipynb` line by line, benchmarks, classical baselines, limits and research

You only need

- vectors, matrices, derivatives
- basic probability
- a little Python

Three kinds of statements

IN REPOSITORY code that exists in this repository

THEORY ONLY explained, not implemented here

ILLUSTRATIVE EXPERIMENT small experiments run for this lecture (scripts in `qml_source/`)

What this repository actually contains

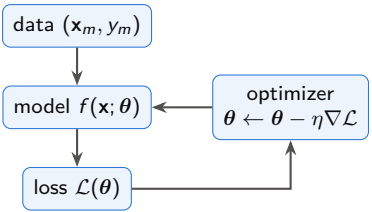
Topic	Status	Where
QCNN classifier (8 qubits, angle encoding, COBYLA)	IN REPOSITORY	`QML.ipynb`
Variational loop (ansatz + classical optimizer)	IN REPOSITORY	`VQE.ipynb`, `QAOA.ipynb`
Amplitude encoding of a vector (`cudaq.qvector(vector)`)	IN REPOSITORY	`HHL.ipynb`, `QFT.ipynb`
Noise channels, density-matrix simulation	IN REPOSITORY	`Noise_Simulation.ipynb`
Multi-GPU target selection (`mgpu`)	IN REPOSITORY	other notebooks (*not* `QML.ipynb`)
Parameter shift, kernel SVM, baselines, plateau / shot / noise / scaling studies	ILLUSTRATIVE EXPERIMENT	`qml_source/make_figures.py`
VQC with entangling feature map, QGAN, qPCA, clustering, quantum RL	THEORY ONLY	—

No GPU, multi-GPU or quantum-hardware run was performed for these slides: every number comes from a CPU simulator.

What is machine learning?

A program **learns** a task from **data** instead of being told the rules.

- **Data**: examples $\mathbf{x}_m$ (features), often with labels y_m
- **Model**: a function $f(\mathbf{x}; \boldsymbol{\theta})$ with parameters $\boldsymbol{\theta}$
- **Loss**: a number $\mathcal{L}(\boldsymbol{\theta})$ that measures how wrong the model is
- **Optimizer**: changes $\boldsymbol{\theta}$ to make $\mathcal{L}$ small
- **Generalisation**: the model must work on *new* data



In `QML.ipynb`

$\mathbf{x} = 8$ pixel values, $y = \pm 1$ (horizontal / vertical line), $f =$ a quantum circuit, $\boldsymbol{\theta} = 12$ angles.

Kinds of learning; classification and regression

Supervised learning

Labelled pairs $(\mathbf{x}_m, y_m)$; learn $\mathbf{x} \mapsto y$.

- **classification**: y from a finite set (spam / not spam, horizontal / vertical)
- **regression**: $y \in \mathbb{R}$ (price, energy)

Unsupervised learning

Only $\mathbf{x}_m$; find structure: **clustering** (k-means), **dimensionality reduction** (PCA), density estimation, generative models.

Reinforcement learning

An agent acts, receives rewards, and learns a policy that maximises the long-term reward.

Outputs

Binary classifier: score $f \in \mathbb{R}$, prediction $\hat{y} = \operatorname{sign} f$ (labels ± 1) or probability $p = \sigma(f)$ (labels 0/1).
Multi-class: softmax $p_k = e^{z_k} / \sum_j e^{z_j}$.
Regression: $\hat{y} = f$ directly.

QML has versions of all three; this repository implements **supervised binary classification**.

Training, validation and test data

- **Training set**: fit the parameters $\boldsymbol{\theta}$.
- **Validation set**: choose hyper-parameters (learning rate, depth, . . .).
- **Test set**: estimate performance on new data — used *once*, at the end.

Data leakage

Fitting a scaler on all data, or picking hyper-parameters by test score, makes the reported accuracy optimistic. Split first; fit everything on the training split.

QML.ipynb IN REPOSITORY

```
train_x, train_y = make_images(20)
test_x, test_y = make_images(20)
```

Two independent draws from the same generator (seed 42): 20 training and 20 test images, half horizontal, half vertical. No validation split: there is no hyper-parameter search.

A test set of 20 images gives an accuracy with a standard error of about ± 8.0 points (slide "uncertainty").

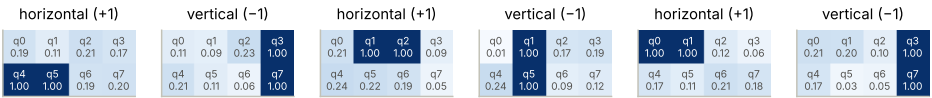
Feature engineering

Features are the numbers the model sees.
Good features can make a hard task easy.

- **raw**: the 8 pixel brightnesses $x_i \in [0, 1]$
- **scaling**: a fixed range, needed for angle encoding ($x \mapsto \pi x$)
- **engineered**: brightest horizontal pair vs. brightest vertical pair

A hand-made rule ILLUSTRATIVE EXPERIMENT

"Horizontal if some horizontal neighbour pair is brighter than every vertical pair." No training: 100 % test accuracy (100.0 % on 2000 fresh images). Lesson: compare quantum models with simple classical methods *on the same features*.



The first six training images of `QML.ipynb` with pixel values; pixel (row r, column c) becomes qubit $4r + c$.

Loss functions

Loss	Formula (one example)	Used for	Note
mean squared error	$(f-y)^2$	regression; `QML.ipynb` with $y=\pm 1$	punishes small margins
binary cross-entropy	$-[y\ln p+(1-y)\ln(1-p)]$	probabilities, $y\in\{0,1\}$	$\partial/\partial z = p-y$ for $p=\sigma(z)$
cross-entropy	$-\ln p_{\text{true}}$, $p=\text{softmax}(\mathbf{z})$	multi-class	
hinge	$\max(0,1-yf)$	SVM	zero once the margin is 1

Worked example: MSE vs. accuracy

Outputs $0.6, -0.2, 0.9, 0.1$, labels $+1, -1, +1, -1$:
errors $-0.4, 0.8, -0.1, 1.1$; MSE $= \frac{0.16+0.64+0.01+1.21}{4} = 0.505$;
signs correct for 3 of 4: accuracy 75 %.

Training objective

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{M}\sum_{m=1}^{M} \ell\Big(f(\mathbf{x}_m;\boldsymbol{\theta}), y_m\Big),$$ the average over the training set. The notebook uses $\ell=(f-y)^2$ with $f=\langle Z_{\text{out}}\rangle \in [-1,1]$.

Gradient descent

The gradient $\nabla\mathcal{L}$ points uphill; step downhill:

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta\,\nabla\mathcal{L}(\boldsymbol{\theta}), \qquad \eta = \text{learning rate.}$$

- η too small: slow; too large: divergence. For a quadratic with curvature $\lambda_{\max}$, stable iff $\eta < 2/\lambda_{\max}$.
- **stochastic** GD: gradient of a random mini-batch.
- **momentum**: $v \leftarrow \beta v + \nabla\mathcal{L}$, $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta v$; Adam adds per-parameter step sizes.
- **gradient-free** methods (COBYLA, Nelder–Mead) use only loss values.

Worked example

$\mathcal{L} = (w_1 - 3)^2 + 2(w_2 + 1)^2$,
start $(0, 0)$, $\eta = 0.1$.
$\nabla\mathcal{L} = (2(w_1 - 3),\ 4(w_2 + 1)) = (-6, 4)$.
New point $(0, 0) - 0.1(-6, 4) = (0.6, -0.4)$.
Loss: $11 \to 5.76 + 0.72 = 6.48$.

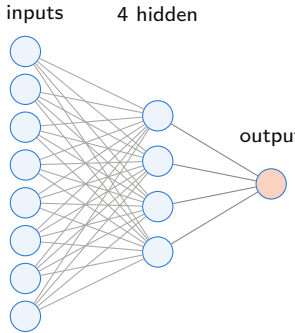
In QML.ipynb

COBYLA (gradient-free, IN REPOSITORY) with 100 loss evaluations; no gradients are computed.

Neural networks

A **neuron**: $a = \phi(\mathbf{w} \cdot \mathbf{x} + b)$ with an activation ϕ.
A **layer**: $\mathbf{h} = \phi(W\mathbf{x} + \mathbf{b})$; layers are stacked.

- activations: sigmoid σ, tanh, ReLU $\max(0, z)$
- derivatives: $\sigma' = \sigma(1 - \sigma)$, $\tanh' = 1 - \tanh^2$, $\mathrm{ReLU}' = [z > 0]$
- without non-linear activations the network is linear and cannot solve XOR
- **CNNs** share the weights of small filters over the image and pool to reduce size — the template for the QCNN



$8 \cdot 4 + 4 + 4 \cdot 1 + 1 = 41$ parameters. The 8-qubit QCNN has 12.

Back-propagation: the chain rule, worked

Network: $\mathbf{h} = \mathrm{ReLU}(W_1\mathbf{x} + \mathbf{b}_1)$,
$p = \sigma(\mathbf{w}_2 \cdot \mathbf{h} + b_2)$, loss $L = -\ln p$ $(y = 1)$.

$$W_1 = \begin{pmatrix} 1 & -1 \\ 2 & 1 \end{pmatrix}, \quad \mathbf{b}_1 = (0, -2),$$

$$\mathbf{w}_2 = (0.5, -1), \quad b_2 = -1.$$

Forward, $\mathbf{x} = (1, -1)$: $\mathbf{a} = (2, -1)$, $\mathbf{h} = (2, 0)$, $z = 0$, $p = \frac{1}{2}$, $L = \ln 2$.

Backward:

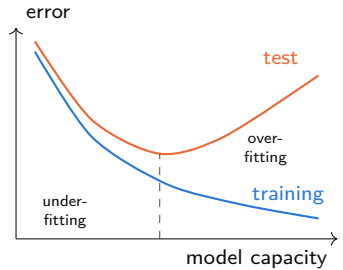
- $\partial L/\partial z = p - y = -0.5$ (sigmoid + cross-entropy)
- $\partial L/\partial \mathbf{w}_2 = -0.5\,\mathbf{h} = (-1, 0)$, $\partial L/\partial b_2 = -0.5$
- $\partial L/\partial a_j = -0.5\, w_{2,j}[a_j > 0] = (-0.25, 0)$
- $\partial L/\partial W_1 = \begin{pmatrix} -0.25 & 0.25 \\ 0 & 0 \end{pmatrix}$: the inactive ReLU gets no gradient

Quantum analogue

A quantum circuit cannot be back-propagated through on hardware; its derivatives come from extra circuit runs (parameter shift, Part C).

Overfitting and underfitting

- **Underfitting** (high bias): training and test errors both high. Remedy: more capacity, better features.
- **Overfitting** (high variance): training error low, test error high. Remedy: regularisation ($+\lambda\|\boldsymbol{\theta}\|^2$), more data, simpler model, early stopping.
- The **generalisation gap** is test error minus training error.
- Regularisation *raises* the training error on purpose to lower the test error.



Recorded QCNN run

training accuracy 90 %, test accuracy 85 %: a small gap — with only 12 parameters there is little room to overfit.

Model evaluation: confusion matrix and metrics

Positive class = "horizontal".

	pred. H	pred. V
true H	TP	FN
true V	FP	TN

- accuracy $= \frac{TP+TN}{\text{all}}$
- precision $= \frac{TP}{TP+FP}$ (how many alarms are right)
- recall $= \frac{TP}{TP+FN}$ (how many positives are found)
- $F_1 = \frac{2PR}{P+R} = \frac{2TP}{2TP+FP+FN}$
- loss (MSE) measures confidence, not only correctness

QCNN test set (re-executed run)

	pred. H	pred. V
true H	9	1
true V	2	8

accuracy = 85 %, precision = 0.818, recall = 0.900, $F_1 = 0.857$, test MSE = 0.7251.

The notebook prints accuracy and loss only; precision, recall and F1 were computed for this lecture from the same outputs.

How certain is an accuracy?

The number of correct test predictions is binomial. The estimated accuracy $\hat{a}$ on M images has

$$\mathrm{SE}(\hat{a}) = \sqrt{\frac{\hat{a}(1-\hat{a})}{M}}.$$

- $M = 20$, $\hat{a} = 0.85$: $\mathrm{SE} = 0.080$ — the 95 % range is roughly 69–100 %.
- $M = 2000$: $\mathrm{SE} \approx 0.009$.

Measured ILLUSTRATIVE EXPERIMENT

The trained QCNN on 2000 fresh images from the same generator: 80.2 % — consistent with 85 ± 8 % on 20 images.

Rules of thumb

- report the test-set size and an error bar
- never compare models on a few dozen images
- fix the seed for reproducibility, but test several seeds for robustness

Classical ML versus quantum ML

	Classical ML	**(Hybrid) quantum ML**
model	neural network, kernel machine, tree, ...	parameterized quantum circuit + classical optimizer
data in the model	numbers in memory	encoded into a quantum state (costs gates)
output	any real number, read directly	expectation values, estimated from *shots*
gradients	back-propagation (one backward pass)	extra circuit runs per parameter (parameter shift)
state space	d numbers	2^n amplitudes, not directly readable
maturity	huge models, GPUs, mature tools	small, noisy devices; mostly simulated today
advantage	—	**not demonstrated** for practical classical data

Hybrid = a quantum circuit inside a classical training loop. This is what `QML.ipynb` implements.

Qubits, superposition and measurement

A qubit state is a unit vector

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle = \begin{pmatrix}\alpha\\ \beta\end{pmatrix}, \quad |\alpha|^2 + |\beta|^2 = 1,$$

$\alpha, \beta \in \mathbb{C}$ are **amplitudes**. **Superposition**: both are non-zero.

Born rule: measuring gives 0 with probability $|\alpha|^2$, 1 with probability $|\beta|^2$; the state collapses.

- global phase $e^{i\phi}\,|\psi\rangle$: unobservable
- relative phase ($|+\rangle$ vs. $|-\rangle$): observable after a change of basis

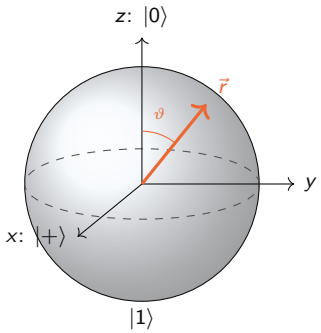
Worked example

$|\psi\rangle = \frac{1}{2}\,|0\rangle + \frac{\sqrt{3}}{2} e^{i\pi/4}\,|1\rangle$:
$P(0) = \frac{1}{4}$, $P(1) = \frac{3}{4}$,
$\langle Z\rangle = P(0) - P(1) = -\frac{1}{2}$,
$\langle X\rangle = 2\,\mathrm{Re}(\alpha^*\beta) = \frac{\sqrt{3}}{2}\cos\frac{\pi}{4} = \frac{\sqrt{6}}{4}$.

Shots

Each run gives one bit. With S shots, $\widehat{\langle Z\rangle}$ has error $\sqrt{(1-\langle Z\rangle^2)/S} \approx 1/\sqrt{S}$. 1000 shots, 620 zeros: $\langle Z\rangle \approx 0.24 \pm 0.03$.

The Bloch sphere



Every pure state can be written

$$|\psi\rangle = \cos\tfrac{\vartheta}{2}\,|0\rangle + e^{i\varphi}\sin\tfrac{\vartheta}{2}\,|1\rangle$$

and drawn as the unit vector

$$\vec{r} = (\langle X\rangle, \langle Y\rangle, \langle Z\rangle)$$

$$= (\sin\vartheta\cos\varphi,\ \sin\vartheta\sin\varphi,\ \cos\vartheta).$$

- gates = rotations of the sphere; $R_Y(\theta)$ turns $\vec{r}$ about the y-axis
- note the **half angle**: $\vartheta = \frac{\pi}{4}$ means amplitudes $\cos\frac{\pi}{8}, \sin\frac{\pi}{8}$
- notebook encoding $R_Y(\pi x)\,|0\rangle$: pixel $x \mapsto$ polar angle πx; dark at the north pole, lit at the south pole

Tensor products: many qubits

n qubits live in $\mathbb{C}^{2^n}$; basis states $|q_0 q_1 \dots q_{n-1}\rangle$ (qubit 0 written first, as CUDA-Q prints them).

$$|a\rangle \otimes |b\rangle = \begin{pmatrix} a_0 b_0 \\ a_0 b_1 \\ a_1 b_0 \\ a_1 b_1 \end{pmatrix}, \qquad A \otimes B = \begin{pmatrix} a_{11} B & a_{12} B \\ a_{21} B & a_{22} B \end{pmatrix}.$$

Worked example

$R_Y(\frac{\pi}{2})\,|0\rangle \otimes R_Y(\pi)\,|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes |1\rangle = \frac{1}{\sqrt{2}}(0, 1, 0, 1)^{\mathsf{T}}$.

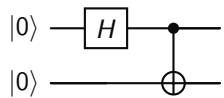
Exponential memory

A state vector stores 2^n complex numbers: $16 \cdot 2^n$ bytes in double precision.

n	memory
8	4 KB (notebook default)
15	0.5 MB
30	16 GiB
32	64 GiB (one 80 GB GPU)
34	256 GiB (four GPUs)

Entanglement

A two-qubit pure state $\sum c_{ab}\,|ab\rangle$ is a **product state** iff $c_{00}c_{11} - c_{01}c_{10} = 0$; otherwise it is **entangled**.



$|\Phi^+\rangle = \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle)$: $\langle Z_0\rangle = \langle Z_1\rangle = 0$, but $\langle Z_0 Z_1\rangle = \langle X_0 X_1\rangle = 1$.

Entanglement hides local information

Apply $R_Y(\theta)$ to qubit 1 of $|\Phi^+\rangle$: its reduced state is $\frac{I}{2}$, so $\langle Z_1\rangle = 0$ for *every* θ, while $\langle Z_0 Z_1\rangle = \cos\theta$. A one-qubit readout can be blind to a parameter — a first glimpse of barren plateaus.

Measuring entanglement

Purity of the reduced state $\operatorname{tr}\rho_1^2 \in [\frac{1}{2}, 1]$: 1 = product, $\frac{1}{2}$ = maximally entangled. For $\cos\frac{\theta}{2}\,|00\rangle + \sin\frac{\theta}{2}\,|11\rangle$: $\operatorname{tr}\rho_1^2 = \frac{1+\cos^2\theta}{2}$.

Quantum gates are unitary matrices

gate	matrix	action
X, Y, Z	Pauli matrices	bit flip, both, phase flip
H	$\frac{1}{\sqrt{2}}\left(\begin{smallmatrix}1 & 1\\ 1 & -1\end{smallmatrix}\right)$	$\lvert 0\rangle \to \lvert +\rangle$; $HZH = X$
S	$\mathrm{diag}(1, i)$	$SXS^\dagger = Y$
$R_P(\theta)$	$e^{-i\theta P/2} = \cos\frac{\theta}{2} I - i \sin\frac{\theta}{2} P$	rotation by θ about axis P
CNOT	$\lvert 0\rangle\langle 0\rvert \otimes I + \lvert 1\rangle\langle 1\rvert \otimes X$	flips the target if control $= 1$
C-U	$\lvert 0\rangle\langle 0\rvert \otimes I + \lvert 1\rangle\langle 1\rvert \otimes U$	e.g. `rz.ctrl`, `rx.ctrl`

Pauli algebra: $X^2 = Y^2 = Z^2 = I$, $XY = iZ = -YX$ (cyclic); so $(aX + bY + cZ)^2 = (a^2 + b^2 + c^2)I$.

Unitary: $U^\dagger U = I$

Columns orthonormal; reversible; preserves the norm. $\frac{1}{\sqrt{2}}\left(\begin{smallmatrix}1 & 1\\ 1 & 1\end{smallmatrix}\right)$ is *not* unitary.

Controlled-R_Z, basis $\lvert 00\rangle, \lvert 01\rangle, \lvert 10\rangle, \lvert 11\rangle$

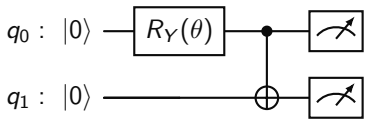
$\mathrm{diag}(1, 1, e^{-i\theta/2}, e^{i\theta/2})$ — not the controlled-phase $\mathrm{diag}(1, 1, 1, e^{i\theta})$: they differ by a relative phase.

Heisenberg picture

$R_Y(\theta)^\dagger Z R_Y(\theta) = \cos\theta\, Z - \sin\theta\, X$.

Quantum circuits and expectation values

Read circuits left to right; each wire is a qubit; boxes are gates; the meter is a measurement.



State: $\cos\frac{\theta}{2}|00\rangle + \sin\frac{\theta}{2}|11\rangle$; $\langle Z_1\rangle = \cos\theta$, $\langle Z_0 Z_1\rangle = 1$.

Expectation value of an observable M (Hermitian):

$$\langle M\rangle = \langle\psi|M|\psi\rangle = \sum_k \lambda_k P(\lambda_k).$$

In CUDA-Q:

- `cudaq.sample(kernel)`: bitstring counts (qubit 0 first)
- `cudaq.observe(kernel, cudaq.spin.z(7), ...)`: $\langle Z_7\rangle$, *exact* on a state-vector simulator, estimated with `shots_count`

From counts

`{00:400, 01:100, 10:300, 11:200}`: $\langle Z_0\rangle = 0$, $\langle Z_1\rangle = 0.4$, $\langle Z_0 Z_1\rangle = 0.2$.

Parameterized quantum circuits

A **parameterized quantum circuit** (PQC) $U(\boldsymbol{\theta})$ contains gates whose angles are trainable numbers. Its model output is

$$f(\boldsymbol{\theta}) = \langle 0|U(\boldsymbol{\theta})^\dagger\, M\, U(\boldsymbol{\theta})|0\rangle\,.$$

Key fact: if θ_k appears in *one* gate $e^{-i\theta_k P/2}$, then

$$f = a + b\cos\theta_k + c\sin\theta_k$$

(period 2π, three numbers determine it).

Worked example

$R_X(\theta_2)R_Y(\theta_1)\,|0\rangle$: Bloch vector $(0,0,1) \xrightarrow{R_Y} (\sin\theta_1, 0, \cos\theta_1) \xrightarrow{R_X}$ $z = \cos\theta_1\cos\theta_2$. So $f(\theta_1,\theta_2) = \cos\theta_1\cos\theta_2$ — trigonometric in each angle separately, not $\cos(\theta_1+\theta_2)$.

Why trigonometric?

$e^{-i\theta P/2}$ has eigenvalues $e^{\mp i\theta/2}$; f multiplies the ket and bra factors, giving frequencies 0 and 1 in θ.

Quantum state preparation

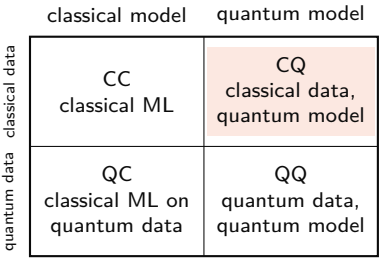
Every QML model starts by **preparing a state** from classical numbers.

- a product of single-qubit rotations: n gates, depth 1 (angle encoding)
- a basis state $|b\rangle$: at most n X gates
- an arbitrary state of n qubits: in general $\Theta(2^n)$ gates (Möttönen-type or Shende–Bullock–Markov constructions)
- simulators can load a vector directly: `cudaq.qvector(vector)` IN REPOSITORY in `HHL.ipynb`, `QFT.ipynb` — not available on hardware in this form

The data-loading bottleneck

If loading N numbers costs $O(N)$ gates, any algorithm that then runs in $O(\log N)$ is dominated by loading. Claims of exponential QML speed-ups usually *assume* a cheap loader (e.g. QRAM), which does not exist at scale today.

What is quantum machine learning? Quantum and classical data



`QML.ipynb` is in the CQ corner.

- **Classical data**: images, tables, text — must be *encoded* into qubits (costly, Part C).
- **Quantum data**: states produced by a quantum process (a sensor, a simulation, a phase of matter) — already quantum; this is where the strongest theoretical advantages are known.
- **Hybrid QML**: a quantum circuit computes features or outputs; a classical computer computes the loss and updates the parameters.

Classical-to-quantum data encoding

Encoding	State	Qubits	Gates	Notes
basis	$\lvert b_1 b_2 \dots b_n \rangle$	1 per bit	$\leq n$ X	binary data only
angle	$\bigotimes_i R_Y(x_i) \lvert 0 \rangle$	1 per feature	d, depth 1	product state; `QML.ipynb` uses $R_Y(\pi x_i)$
amplitude	$\frac{1}{\lVert \mathbf{x} \rVert} \sum_j x_j \lvert j \rangle$	$\lceil \log_2 d \rceil$	$O(d)$ in general	exponentially compact
feature map	$U_\Phi(\mathbf{x}) \lvert 0 \rangle^{\otimes n}$	1 per feature	problem dependent	entangling (e.g. ZZ map)
re-uploading	$\prod_l W_l S(\mathbf{x})$	any	L copies of $S(\mathbf{x})$	richer functions of $\mathbf{x}$

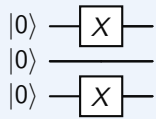
Resource example

$d = 256$ features of 8 bits each: basis encoding 2048 qubits, angle encoding 256 qubits, amplitude encoding 8 qubits. Fewer qubits $\Rightarrow$ deeper state preparation.

Basis and amplitude encoding: worked examples

Basis encoding

$5 = 101_2 \mapsto |101\rangle$: apply X to qubits 0 and 2.



A superposition of many data strings can be prepared, but reading out results needs many measurements.

Amplitude encoding

$\mathbf{x} = (1, 2, 4, 2)$, $\|\mathbf{x}\| = 5$:

$$|\psi_{\mathbf{x}}\rangle = 0.2\,|00\rangle + 0.4\,|01\rangle + 0.8\,|10\rangle + 0.4\,|11\rangle\,.$$

2 qubits for 4 numbers.
$P(q_0 = 1) = 0.8^2 + 0.4^2 = 0.80$;
$P(q_1 = 1) = 0.32$.
Only $|x_j|^2/\|\mathbf{x}\|^2$ can be sampled; the norm is lost and must be stored classically.

Angle encoding — the encoding of `QML.ipynb` IN REPOSITORY

Pixel $x_i \in [0,1] \to$ qubit i:

$$R_Y(\pi x_i)\,|0\rangle = \cos\tfrac{\pi x_i}{2}\,|0\rangle + \sin\tfrac{\pi x_i}{2}\,|1\rangle\,,$$

$$|\psi(\mathbf{x})\rangle = \bigotimes_{i=0}^{7} R_Y(\pi x_i)\,|0\rangle\,.$$

- dark $x = 0 \to |0\rangle$, lit $x = 1 \to |1\rangle$
- $\langle Z_i \rangle = \cos(\pi x_i)$
- 8 gates, depth 1, no entanglement yet

Worked example

$x = \frac{1}{3}$: half-angle $\frac{\pi}{6}$; $P(1) = \sin^2\frac{\pi}{6} = \frac{1}{4}$, $\langle Z\rangle = \frac{1}{2}$.
$x = \frac{1}{4}$: $\cos\frac{\pi}{8}\,|0\rangle + \sin\frac{\pi}{8}\,|1\rangle$, Bloch vector $(\frac{1}{\sqrt{2}}, 0, \frac{1}{\sqrt{2}})$.

```
for i in range(len(pixels)):
    ry(np.pi * pixels[i], qubits[i])
```

Row-by-row flattening: pixel (r, c) of the 2×4 image is qubit $4r + c$.

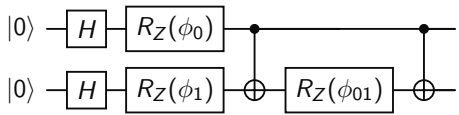
Quantum feature maps

A **feature map** $\mathbf{x} \mapsto \rho(\mathbf{x}) = |\psi(\mathbf{x})\rangle\langle\psi(\mathbf{x})|$ sends data into a 2^n-dimensional space of states. Linear models in that space = non-linear models in $\mathbf{x}$ (as with the XOR problem).

ZZ feature map (Havlíček et al. 2019) THEORY ONLY:

$$U_\Phi(\mathbf{x}) = \exp\left(i\sum_i \phi_i(\mathbf{x})Z_i + i\sum_{i<j}\phi_{ij}(\mathbf{x})Z_iZ_j\right)H^{\otimes n},$$

applied twice, e.g. $\phi_i = x_i$, $\phi_{ij} = (\pi - x_i)(\pi - x_j)$.



(one repetition, two qubits; ZZ phase via CNOT–R_Z–CNOT; angle conventions differ by factors of 2)

Product vs. entangling maps

Angle encoding is a product map: its kernel factorises and is classically trivial. Entangling maps can produce kernels that are believed hard to compute classically — which still does not guarantee that they are *useful*.

Quantum kernels

Fidelity (quantum) kernel:

$$k(\mathbf{x}, \mathbf{x}') = |\langle\psi(\mathbf{x})|\psi(\mathbf{x}')\rangle|^2 = \mathrm{tr}\left[\rho(\mathbf{x})\rho(\mathbf{x}')\right].$$

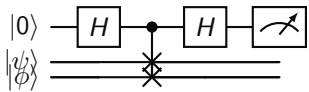
Always symmetric and positive semi-definite (exactly; shot-noise estimates may not be).

Notebook encoding

$k(\mathbf{x}, \mathbf{x}') = \prod_i \cos^2 \frac{\pi(x_i - x_i')}{2}$; e.g. $\mathbf{x} = (1, 0.5)$, $\mathbf{x}' = (0.5, 0)$: $k = \frac{1}{2} \cdot \frac{1}{2} = \frac{1}{4}$. A closed classical formula!

Estimating k on a quantum computer:

- **compute–uncompute**: run $U(\mathbf{x}')^\dagger U(\mathbf{x})$; $k = P(0 \ldots 0)$
- **swap test**: $P(\text{ancilla} = 0) = \frac{1}{2}(1 + k)$

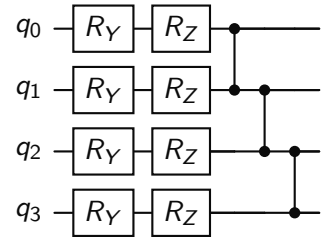


M training points need $\binom{M}{2}$ kernel circuits (190 for $M = 20$).

Variational quantum circuits (ansatz)

A **variational circuit** (ansatz) $U(\boldsymbol{\theta})$: layers of trainable rotations and fixed entangling gates.

- **hardware-efficient ansatz** (HEA): R_Y, R_Z on every qubit + a CNOT/CZ ladder, repeated L times — used in `VQE.ipynb` IN REPOSITORY
- **problem-inspired** ansätze: UCC (chemistry), QAOA (optimization), QCNN (translation-invariant data)
- more layers $\Rightarrow$ more expressive, but harder to train and noisier



One HEA layer ($2n$ parameters) as used in the barren-plateau experiment of Part E.

Quantum neural networks and data re-uploading

A **quantum neural network** (QNN) = encoding + variational circuit + measurement, trained like a neural network:

$$f(\mathbf{x};\boldsymbol{\theta}) = \langle \psi(\mathbf{x}) | U(\boldsymbol{\theta})^\dagger M U(\boldsymbol{\theta}) | \psi(\mathbf{x}) \rangle \,.$$

What functions can it represent? With encoding gates $e^{-ixP/2}$ used L times,

$$f(x) = \sum_{\omega=-L}^{L} c_\omega(\boldsymbol{\theta})\, e^{i\omega x},$$

a Fourier series of degree L (Schuld, Sweke & Meyer 2021).

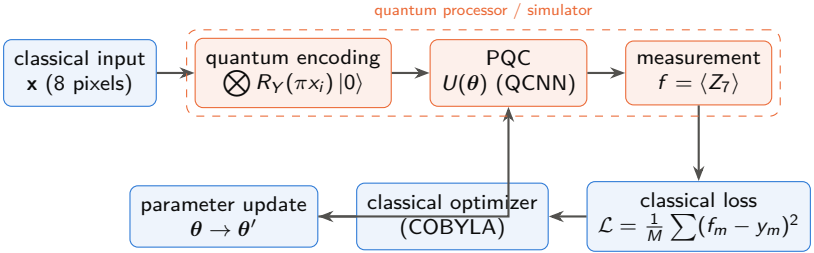
Consequences

- one encoding: $f = a + b\cos x + c\sin x$ only
- repeat ("re-upload") the data to reach higher frequencies
- the trainable part controls the coefficients c_ω

QML.ipynb

Each pixel is encoded once ($L = 1$ per pixel), so f is a polynomial of degree 1 in each $\cos \pi x_i$, $\sin \pi x_i$.

Hybrid quantum–classical learning: the full pipeline



1. scale the features to angles and prepare $|\psi(\mathbf{x}_m)\rangle$;
2. apply $U(\boldsymbol{\theta})$ and estimate $f(\mathbf{x}_m;\boldsymbol{\theta})$ (exactly on a simulator, from shots on hardware);
3. repeat for all M images and compute $\mathcal{L}(\boldsymbol{\theta})$ classically;
4. the optimizer proposes new $\boldsymbol{\theta}$; repeat until the budget is used.

Cost per loss evaluation: M circuits. Notebook: $100 \times 20 + 20 = 2020$ circuits, ≈ 1.7 ms each on `qpp-cpu` $\Rightarrow\approx 3.4$ s (recorded).

Loss, measurement and the classifier output

The quantum model returns an **expectation value** $f = \langle Z_{\text{out}} \rangle \in [-1, 1]$:

- prediction $\hat{y} = \operatorname{sign} f$ (decision boundary $f = 0$)
- margin $|f|$ = confidence
- loss $\mathcal{L} = \frac{1}{M} \sum (f_m - y_m)^2$, so a correct prediction with $|f| = 0.2$ still costs $(1 - 0.2)^2 = 0.64$

Recorded run

training MSE 0.6704 at training accuracy 90 %: mean $|\langle Z_7 \rangle| = 0.197$ — correct signs, small margins.

Other read-outs:

- $p(y = 1) = \frac{1+f}{2}$ with cross-entropy
- parity of several qubits, $\langle Z_0 Z_1 \rangle$
- one qubit per class, softmax over $\langle Z_k \rangle$
- trainable classical post-processing $wf + b$

Measurement cost

On hardware f is estimated from S shots with error $\approx 1/\sqrt{S}$: margins of 0.2 need well over 25 shots per image.

Parameter optimization: gradient-free and gradient-based

	gradient-free (COBYLA, Nelder–Mead, Powell, SPSA*)	**gradient-based** (GD, Adam, L-BFGS)
needs	loss values only	$\nabla\mathcal{L}$: parameter shift or finite differences
cost per step	1 loss evaluation (M circuits)	$\approx 2\times$(number of gate occurrences)$\times M$ circuits
scaling with #params	poor beyond a few dozen	good if gradients are cheap
shot noise	COBYLA is sensitive; SPSA is designed for it	averages over many circuits
in this repository	`QML.ipynb`: COBYLA (default), Nelder–Mead, Powell IN REPOSITORY	L-BFGS-B with finite differences in `VQE.ipynb`, `QAOA.ipynb` IN REPOSITORY

*SPSA: simultaneous perturbation, 2 evaluations per step for any number of parameters (theory here).

Cost of exact gradients for the 8-qubit QCNN

11 convolution blocks $\times$ 2 R_Y $\times$ 2 circuits + 7 pooling blocks $\times$ 2 controlled rotations $\times$ 4 circuits = 100 circuits per image, 2000 per gradient over 20 images — as many as the whole COBYLA run.

The parameter-shift rule: derivation

Gate $G(\theta) = e^{-i\theta P/2}$ with $P^2 = I$ (a Pauli). Then $f(\theta) = a + b\cos\theta + c\sin\theta$, and

$$\begin{aligned} f(\theta + \tfrac{\pi}{2}) - f(\theta - \tfrac{\pi}{2}) &= b[\cos(\theta + \tfrac{\pi}{2}) - \cos(\theta - \tfrac{\pi}{2})] \\ &\quad + c[\sin(\theta + \tfrac{\pi}{2}) - \sin(\theta - \tfrac{\pi}{2})] \\ &= -2b\sin\theta + 2c\cos\theta = 2f'(\theta). \end{aligned}$$

$$\boxed{f'(\theta) = \tfrac{1}{2}\left[f(\theta + \tfrac{\pi}{2}) - f(\theta - \tfrac{\pi}{2})\right]}$$

Properties

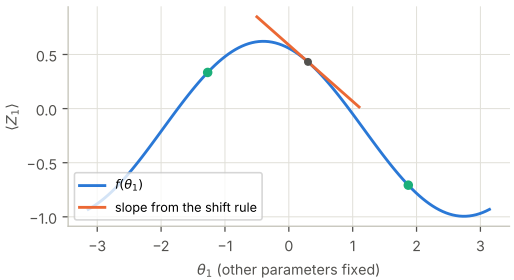
- **exact**: no step-size error (unlike finite differences)
- runs on hardware: two ordinary circuits
- shot noise remains: each f is estimated
- **only** for generators with two eigenvalues $\pm\frac{1}{2}$; shared parameters need one shift pair *per occurrence*; controlled rotations need four terms

Numbers

$f(\theta + \frac{\pi}{2}) = 0.30$, $f(\theta - \frac{\pi}{2}) = -0.50 \Rightarrow$ $f'(\theta) = 0.40$.

Parameter shift, checked with CUDA-Q

ILLUSTRATIVE EXPERIMENT



2-qubit VQC: $R_Y(x_0) \otimes R_Y(x_1)$, $R_Y(\theta_1) \otimes R_Y(\theta_2)$, CNOT, $R_Y(\theta_3)$ on q_1; $f = \langle Z_1 \rangle$; $\mathbf{x} = (0.4, 1.1)$.

At $\boldsymbol{\theta} = (0.3, -0.7, 0.5)$, two `cudaq.observe` calls per parameter give

$$\nabla f = (-0.5207,\ -0.7030,\ -0.6795).$$

Largest difference from a central finite difference ($h = 10^{-5}$): $3.5e-11$ — the shift rule is exact.

Hybrid gradient of the loss

$\frac{\partial \mathcal{L}}{\partial \theta_k} = \frac{1}{M} \sum_m 2(f_m - y_m) \frac{\partial f_m}{\partial \theta_k}$: quantum part by shift, classical part by the chain rule.

When the two-term rule fails ILLUSTRATIVE EXPERIMENT

Shared parameters: in the QCNN, $\theta_{1,1}$ sits in 7 R_Y gates, so f contains frequencies up to 7 in it. **Controlled rotations**: generator $|1\rangle\langle 1| \otimes \frac{P}{2}$ has eigenvalues $0, \pm\frac{1}{2}$ (frequencies $\frac{1}{2}$, 1).

Naive two-term rule on the *trained* QCNN, first training image:

parameter	exact ∂f	two-term shift
$\theta_{1,1}$ (shared R_Y)	$+0.2890$	$+0.0094$
$\theta_{1,3}$ (controlled R_Z)	-0.0055	$+0.0945$
$\theta_{1,4}$ (controlled R_X)	-0.0997	-0.0089

Four-term rule (controlled rotations)

$$f' = d_+\left[f(\theta+\tfrac{\pi}{2}) - f(\theta-\tfrac{\pi}{2})\right] - d_-\left[f(\theta+\tfrac{3\pi}{2}) - f(\theta-\tfrac{3\pi}{2})\right],$$

$d_\pm = \frac{\sqrt{2}\pm 1}{4\sqrt{2}}$.

Hadamard test, $f(\theta) = \cos\frac{\theta}{2}$, $\theta = 0.9$

exact -0.2175; two-term -0.3076 (wrong); four-term -0.2175 (exact).

Correct recipe for the QCNN: shift every occurrence separately and add (product rule).

Variational Quantum Classifier (VQC) THEORY ONLY

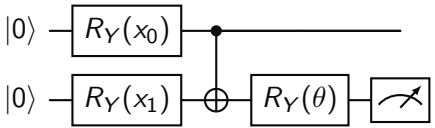
Idea: feature map → trainable ansatz → measure one observable → threshold.

$$f(\mathbf{x};\boldsymbol{\theta}) = \langle\psi(\mathbf{x})|U^\dagger(\boldsymbol{\theta})\, M\, U(\boldsymbol{\theta})|\psi(\mathbf{x})\rangle\,,$$

$$\hat{y} = \operatorname{sign}\big(f(\mathbf{x};\boldsymbol{\theta}) + b\big).$$

Trained by minimising MSE / cross-entropy with a classical optimizer. The QCNN of the notebook is a VQC with a special ansatz; a general VQC with an entangling feature map is *not* in this repository.

- use: small classification tasks, quantum data
- limits: trainability, shot cost, no proven advantage on classical data



Worked example (Heisenberg picture)

$Z_1 \xrightarrow{R_Y(\theta)} \cos\theta Z_1 - \sin\theta X_1 \xrightarrow{\text{CNOT}}$
$\cos\theta Z_0 Z_1 - \sin\theta X_1$:
$f = \cos\theta\cos x_0\cos x_1 - \sin\theta\sin x_1$.
$\mathbf{x} = (0, \frac{\pi}{3})$, $\theta = \frac{\pi}{6}$: $f = 0$; $f(\theta \pm \frac{\pi}{2}) = \mp 1$;
$f' = -1$; $y = +1$:
$\partial_\theta (f-y)^2 = 2(0-1)(-1) = 2$.

Quantum SVM and kernel methods THEORY ONLY

SVM: find the maximum-margin separator in feature space. Dual problem (only kernels appear):

$$\max_{\boldsymbol{\alpha}} \sum_m \alpha_m - \tfrac{1}{2} \sum_{m,m'} \alpha_m \alpha_{m'} y_m y_{m'} k(\mathbf{x}_m, \mathbf{x}_{m'})$$

subject to $0 \le \alpha_m \le C$, $\sum_m \alpha_m y_m = 0$.
Prediction: $\hat{y} = \operatorname{sign}\big(\sum_m \alpha_m y_m k(\mathbf{x}_m, \mathbf{x}) + b\big)$.

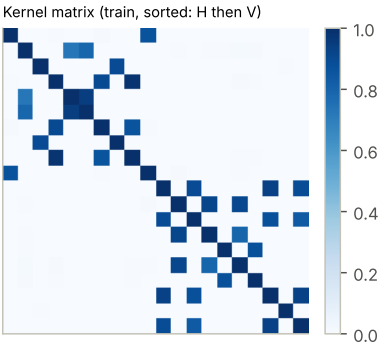
Quantum SVM: the quantum computer only estimates k; the optimisation is a classical convex QP — no barren plateaus.

Strengths and limits

- convex training, guaranteed optimum
- $O(M^2)$ kernel circuits, each with many shots
- kernel values concentrate exponentially for many features (Part E) $\Rightarrow$ poor generalisation
- a kernel that is classically computable gives no quantum advantage

Havlíček et al., *Nature* 567 (2019); Schuld & Killoran, PRL 122 (2019).

Quantum-kernel SVM on the notebook's data ILLUSTRATIVE EXPERIMENT



Feature map = the notebook's angle encoding; kernel from CUDA-Q states (`cudaq.get_state`); SVM dual solved classically ($C = 10$).

- CUDA-Q kernel = analytic $\prod \cos^2$ formula to $1.1e-15$
- mean off-diagonal kernel value 0.084 (8 features already push k towards 0)
- 17 support vectors of 20; training accuracy 100 %, test accuracy 100 %
- 2000 fresh images: 95.2 % (QCNN: 80.2 %)

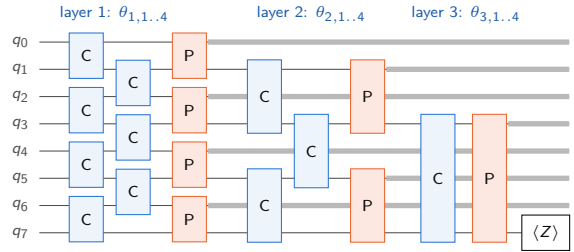
Interpretation

A better classifier than the QCNN here — but its kernel is a cheap classical formula, so this is *not* a quantum advantage.

QNN and the Quantum Convolutional Neural Network IN REPOSITORY

QCNN (Cong, Choi & Lukin, *Nat. Phys.* 2019):

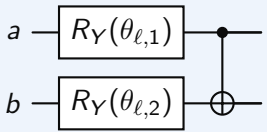
- **convolution**: the same 2-qubit block on all neighbouring pairs (shared weights)
- **pooling**: a controlled rotation passes information from a source to a sink; the source is discarded
- repeat until one qubit is left: $\lceil \log_2 N \rceil$ layers, $O(\log N)$ parameters
- provably free of barren plateaus (Pesah et al., PRX 2021)



8 qubits $\to 4 \to 2 \to 1$. C = convolution block, P = pooling block (grey = discarded qubit). Output: qubit 7.

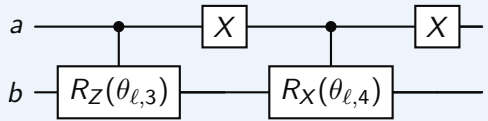
QCNN building blocks (`QML.ipynb`)

Convolution block on (a, b), layer ℓ



$U_\ell = \mathrm{CNOT}_{a\to b}\big(R_Y(\theta_{\ell,1}) \otimes R_Y(\theta_{\ell,2})\big)$, on pairs $(0,1),(2,3),\dots$ then $(1,2),(3,4),\dots$

Pooling block, source $a \to$ sink b



$P_\ell = |1\rangle\langle 1|_a \otimes R_Z(\theta_{\ell,3}) + |0\rangle\langle 0|_a \otimes R_X(\theta_{\ell,4})$

- The $X \cdots X$ sandwich turns the second controlled gate into an *anti*-controlled one (acts when $a = 0$).
- **Deferred measurement**: "measure a, then rotate b depending on the result" = controlled rotation, then ignore a — no mid-circuit measurement needed.
- Gate count: $N + 3n_{\text{conv}} + 4n_{\text{pool}}$; $N = 8$: $8 + 33 + 28 = 69$ gates, depth 21.

QCNN size versus number of qubits IN REPOSITORY

N	layers	params	conv	pool	output
4	2	8	4	3	3
8	3	12	11	7	7
15	4	16	25	14	14
16	4	16	26	15	15
20	5	20	35	19	19
30	5	20	54	29	29

computed with the notebook's `qcnn_layout`; layers $= \lceil \log_2 N \rceil$.

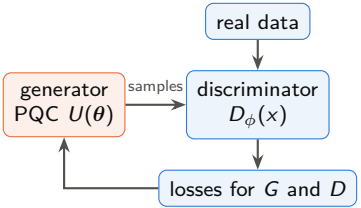
- pooling keeps $\lceil n/2 \rceil$: $20 \to 10 \to 5 \to 3 \to 2 \to 1$
- parameters grow *logarithmically*: 12 at $N = 8$, 20 at $N = 30$
- gates grow *linearly*: $N = 16$: $16 + 3 \cdot 26 + 4 \cdot 15 = 154$
- the classical simulation cost grows *exponentially* (2^N)

Quantum Generative Adversarial Networks THEORY ONLY

A **generator** G produces samples; a **discriminator** D tries to tell them from real data.

$$\min_G \max_D \; \mathbb{E}_{\text{data}}[\ln D(x)] + \mathbb{E}_G[\ln(1 - D(x))].$$

Quantum GAN: G is a PQC whose measurement distribution $p_{\boldsymbol{\theta}}(x) = |\langle x|U(\boldsymbol{\theta})|0\rangle|^2$ is the model; D may be classical or quantum. At equilibrium $p_G = p_{\text{data}}$, $D = \frac{1}{2}$, value $-\ln 4$.



Uses and limits

loading distributions (e.g. for finance), learning quantum states. Training is unstable (as for classical GANs); sampling cost; no general advantage shown.

Quantum Principal Component Analysis THEORY ONLY

Lloyd, Mohseni & Rebentrost (*Nat. Phys.* 2014):

1. prepare many copies of the (normalised) covariance matrix as a density matrix ρ
2. simulate $e^{-i\rho t}$ by repeated partial SWAPs with fresh copies (density-matrix exponentiation)
3. phase estimation $\Rightarrow \sum_k \sqrt{\lambda_k}\,|v_k\rangle\,|\tilde{\lambda}_k\rangle$: large eigenvalues and eigenvectors *as quantum states*

Cost polylogarithmic in the dimension — for low-rank ρ and efficient state access.

Caveats

- reading out an eigenvector classically costs its dimension
- needs QRAM-like data access
- "dequantized" classical sampling algorithms (Tang 2019; Chia et al. 2020) reach polylog scaling for low-rank PCA under comparable access
- needs fault-tolerant hardware (phase estimation)

Classical PCA example: $C = \left(\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right)$, $\lambda = 3, 1$, first component $(1,1)/\sqrt{2}$ explains 75 %.

Quantum clustering and quantum reinforcement learning THEORY ONLY

Clustering

k-means: assign points to the nearest centroid, update centroids. Quantum versions (q-means, Kerenidis et al. 2019) estimate distances with overlap circuits:

$$\|\mathbf{a}-\mathbf{b}\|^2 = 2 - 2\,\mathbf{a}\cdot\mathbf{b}, \quad |\mathbf{a}\cdot\mathbf{b}|^2 = 2P_{\text{swap}}(0) - 1.$$

Example: $P(0) = 0.9 \Rightarrow \mathbf{a}\cdot\mathbf{b} = \sqrt{0.8} = 0.894$, distance 0.46. Kernel-based clustering can use quantum kernels. Same caveats: data access, shot cost, dequantization.

Reinforcement learning

An agent in state s takes action a, gets reward r. Q-learning:

$$Q(s,a) \leftarrow Q + \alpha\Big[r + \gamma \max_{a'} Q(s',a') - Q\Big].$$

$Q = 2$, $\alpha = 0.5$, $r = 1$, $\gamma = 0.9$, $\max Q' = 4$: $Q \to 3.3$.

Quantum RL: a PQC represents the policy $\pi(a \mid s) = |\langle a|U(s;\boldsymbol{\theta})|0\rangle|^2$ or the Q-function (Jerbi et al. 2021; Skolik et al. 2022); speed-ups proven only for specially constructed environments.

Quantum data classification

The original QCNN task: given a quantum state $|\psi\rangle$ produced by a physical system, decide its **phase** (e.g. symmetry-protected topological vs. trivial).

- no encoding cost: the data are already quantum
- the classical alternative must first learn the state from measurements (tomography: exponential in general)
- here quantum models have provable sample-complexity advantages for some tasks (Huang et al., *Science* 2022)

`QML.ipynb` by contrast

classifies *classical* images (horizontal vs. vertical lines) that are angle-encoded — a teaching example of the architecture, explicitly not intended to beat classical methods (notebook, Section 1).

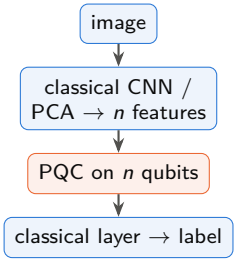
Take-away

The most credible routes to quantum advantage in ML involve **quantum data**, not small classical datasets.

Hybrid quantum–classical architectures

- **quantum layer in a classical network**: classical pre-processing (e.g. CNN features, PCA to n numbers) $\rightarrow$ PQC $\rightarrow$ classical head
- **quantum feature extractor**: measure many observables, train a classical model on them
- **quantum kernel + classical SVM**
- **end-to-end PQC** with classical optimizer — the design of `QML.ipynb`

Frameworks connect CUDA-Q kernels to PyTorch-style autodiff by supplying gradients from parameter shift or adjoint simulation.



With such pipelines it is essential to check how much of the accuracy comes from the classical parts.

Barren plateaus and vanishing gradients

Barren plateau: for random parameters $\mathbb{E}[\partial_k C] = 0$ and

$$\mathrm{Var}[\partial_k C] \in O(b^{-n}), \;\; b > 1.$$

By Chebyshev, gradients larger than ϵ are exponentially rare; resolving them needs $\sim b^n$ shots.

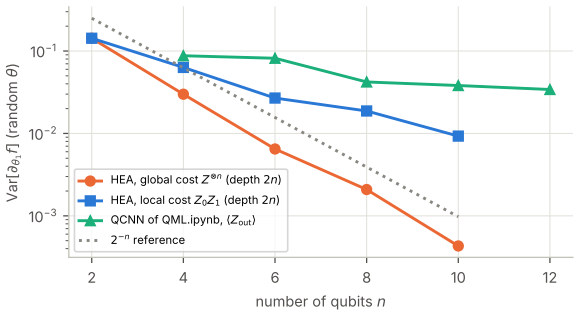
- deep random circuits (2-designs): McClean et al., *Nat. Commun.* 2018
- global costs even in shallow circuits: Cerezo et al., *Nat. Commun.* 2021
- noise-induced plateaus: Wang et al., *Nat. Commun.* 2021
- gradient-free optimizers are affected too (Arrasmith et al. 2021)

Mitigations

- local cost functions, shallow circuits
- structured ansätze with few parameters (QCNN: no barren plateau, Pesah et al. 2021)
- small-angle / identity-block initialisation, layer-wise training
- caution: architectures provably free of plateaus are often classically simulable (Cerezo et al. 2023)

Barren plateaus, measured with CUDA-Q

ILLUSTRATIVE EXPERIMENT



Variance of $\partial f / \partial \theta_1$ over random parameters (200 samples per point; QCNN: 100, random pixels):

- HEA, depth $2n$, global $Z^{\otimes n}$: falls roughly like 2^{-n} (4.3e-04 at $n = 10$)
- same circuit, local $Z_0 Z_1$: decays more slowly (9.3e-03)
- QCNN of `QML.ipynb`: 3.4e-02 at $n = 12$

Small sizes only (CPU simulation); a trend, not a proof.

Expressibility, entanglement, depth and trainability

- **Expressibility**: how much of the state space an ansatz can reach (Sim et al. 2019). More expressive ansätze have flatter landscapes (Holmes et al. 2022): a trade-off.
- **Entanglement**: needed for non-trivial models, but too much entanglement makes local observables uninformative (Part B example).
- **Depth**: more layers $\Rightarrow$ more expressive, more noise, more plateau-prone; parameter count also grows.
- **Inductive bias**: an architecture matched to the data (translation invariance $\rightarrow$ QCNN) beats generic expressivity.

The notebook's QCNN

- depth 21 (8 qubits), only 12 shared parameters
- log-depth in layers, local structure
- README: a 15-qubit run with 60 COBYLA evaluations reached only 69 % — deeper models need more optimizer steps

Noise in QML

ILLUSTRATIVE EXPERIMENT

Real devices apply noise channels, e.g. depolarising $\rho \mapsto (1-p)\rho + p\frac{I}{2}$ after gates. Effects:

- expectation values shrink towards 0; a simple model gives $(1-p)^G$: $0.99^{69} \approx 0.50$
- signs (predictions) survive longer than margins
- noise can flatten landscapes (noise-induced plateaus)

Measured: trained QCNN, 20 test images, depolarising channels after every gate (`density-matrix-cpu`).

p	accuracy	mean $\lvert\langle Z\rangle\rvert$	MSE
0	85 %	0.171	0.725
0.01	85 %	0.112	0.809
0.03	85 %	0.047	0.913
0.1	85 %	0.003	0.995

Accuracy unchanged, but the margins collapse: at $p = 0.1$ the outputs (≈ 0.003) would need $\sim 10^5$ shots per image to read their signs.

Finite shots ILLUSTRATIVE EXPERIMENT

On hardware $\langle Z_7 \rangle$ is estimated from S shots. Test accuracy of the trained QCNN with `cudaq.observe(..., shots_count=S)`, 5 repetitions:

S	mean	range	$1/\sqrt{S}$
10	63 %	60–70 %	0.316
100	86 %	75–100 %	0.100
1,000	84 %	80–90 %	0.032
10,000	83 %	80–85 %	0.010

Why so many shots?

The mean $|\langle Z_7 \rangle|$ on the test images is 0.171: the statistical error must be well below that. Training with shots is harder still: COBYLA then sees a noisy loss.

Exact (state-vector) accuracy: 85 %. Differences of a few points at large S are sampling fluctuations of individual images with $|\langle Z \rangle| \ll 0.1$.

Evaluating a quantum model: classical baselines

ILLUSTRATIVE EXPERIMENT

Model (same 20 training images)	test, 20 images	2000 fresh images	trainable parameters
QCNN of `QML.ipynb` (re-executed)	85 %	80.2 %	12
logistic regression on the 8 pixels	45 %	46.5 %	9
SVM, kernel of the notebook's angle encoding	100 %	95.2 %	20 dual coefficients
hand-made rule (brightest pair)	100 %	100.0 %	0

- Logistic regression fails: the clean images are *not linearly separable* (the two middle horizontal pairs and the two middle vertical lines light the same four pixels).
- The QCNN is non-linear in the pixels and does learn, but generalises worse than the classically computable kernel SVM.

Conclusion

No quantum advantage on this task. The notebook's goal is to teach the QCNN, not to beat classical methods — and it says so.

Dataset preprocessing and the training/testing pipeline IN REPOSITORY

```
rng = np.random.default_rng(SEED)
def make_images(count):
    images, labels = [], []
    for k in range(count):
        image = rng.uniform(0, NOISE_LEVEL,
                            size=(IMAGE_ROWS, IMAGE_COLS))
        if k % 2 == 0: # horizontal
            row = rng.integers(IMAGE_ROWS)
            col = rng.integers(IMAGE_COLS - LINE_LENGTH + 1)
            image[row, col:col + LINE_LENGTH] = 1.0
            labels.append(+1)
        else: # vertical
            col = rng.integers(IMAGE_COLS)
            row = rng.integers(IMAGE_ROWS - LINE_LENGTH + 1)
            image[row:row + LINE_LENGTH, col] = 1.0
            labels.append(-1)
        images.append(image.ravel()) # pixel -> qubit
    return np.array(images), np.array(labels)
```

- synthetic data: dark background in $[0, 0.25]$, one line of 2 pixels $= 1.0$
- classes alternate $\Rightarrow$ exactly balanced
- values already in $[0, 1]$: no further scaling; angle $= \pi x$
- `ravel()`: row-major flattening, pixel $(r, c) \to$ qubit $4r + c$
- train and test: two consecutive calls on one generator, seed 42 $\Rightarrow$ reproducible, independent
- own data: any array of shape (M, N) in $[0, 1]$ with labels ± 1

The structure of QML.ipynb

Section	Content
1	problem statement and mathematics
2	imports, versions, number of GPUs
3	**input parameters** (the only cell to edit)
4	image dataset (`make_images`)
5	target, `qcnn_layout`, kernel, drawing, resources
6	expected output, time per circuit, memory
7	training (execution cell)
8	plots: curves, gallery, histogram
9	final report

Generated by `dev/notebook_builders/build_qml.py`; deliberately short, without the shared monitoring utilities of the other notebooks.

```
IMAGE_ROWS = 2; IMAGE_COLS = 4 # N = 8 qubits
LINE_LENGTH = 2; NOISE_LEVEL = 0.25
TRAIN_IMAGES = 20; TEST_IMAGES = 20
OPTIMIZER = "COBYLA"; MAX_ITERATIONS = 100
INITIAL_PARAMETER_SCALE = 3.14
TARGET = "auto"; SEED = 42
N_QUBITS = IMAGE_ROWS * IMAGE_COLS
```

`TARGET = "auto"`: `"nvidia"` if `cudaq.num_available_gpus() > 0`, else `"qpp-cpu"`. GPU targets are refused when no GPU is visible.

The layout function, line by line (Section 5)

```
def qcnn_layout(n):
    kinds, qubit_a, qubit_b, layers = [], [], [], []
    active, layer = list(range(n)), 0
    while len(active) > 1:
        for start in (0, 1): # conv pairs
            for i in range(start, len(active) - 1, 2):
                kinds.append(0); qubit_a.append(active[i])
                qubit_b.append(active[i + 1]); layers.append(layer)
        kept = []
        for i in range(0, len(active) - 1, 2): # pooling
            kinds.append(1); qubit_a.append(active[i])
            qubit_b.append(active[i + 1]); layers.append(layer)
            kept.append(active[i + 1]) # sink survives
        if len(active) % 2 == 1:
            kept.append(active[-1]) # odd qubit moves on
        active, layer = kept, layer + 1
    return kinds, qubit_a, qubit_b, layers, active[0], layer
```

- plain Python, run once: a list of operations (kind, qubit a, qubit b, layer)
- `start = 0`: pairs $(0,1),(2,3),\ldots$; `start = 1`: $(1,2),(3,4),\ldots$ — like a stride-2 convolution applied twice
- pooling: source `active[i]`, sink `active[i+1]`; sinks survive
- $N = 8$: layer 1: 7 conv + 4 pool on $\{0..7\}$; layer 2: $3 + 2$ on $\{1,3,5,7\}$; layer 3: $1 + 1$ on $\{3,7\}$
- result: 18 blocks, 3 layers, output qubit 7

The CUDA-Q kernel, line by line (Section 5)

```
@cudaq.kernel
def qcnn(pixels: list[float], theta: list[float],
         kinds: list[int], qubit_a: list[int],
         qubit_b: list[int], layers: list[int]):
    qubits = cudaq.qvector(len(pixels)) # |0...0>
    for i in range(len(pixels)):
        ry(np.pi * pixels[i], qubits[i]) # encoding
    for k in range(len(kinds)):
        t = 4 * layers[k] # theta[t..t+3]
        a = qubit_a[k]
        b = qubit_b[k]
        if kinds[k] == 0: # convolution
            ry(theta[t], qubits[a])
            ry(theta[t + 1], qubits[b])
            x.ctrl(qubits[a], qubits[b])
        else: # pooling a -> b
            rz.ctrl(theta[t + 2], qubits[a], qubits[b])
            x(qubits[a])
            rx.ctrl(theta[t + 3], qubits[a], qubits[b])
            x(qubits[a])
```

- `@cudaq.kernel`: compiled (MLIR/QIR) and JIT-compiled on first call
- the kernel only *walks* the operation list; the architecture lives in `qcnn_layout`
- `theta[4ℓ..4ℓ+3]` $= \theta_{\ell+1,1..4}$: weight sharing within a layer
- `x.ctrl(a, b)` = CNOT; `rz.ctrl`, `rx.ctrl` = controlled rotations
- no measurement in the kernel: `cudaq.observe` computes $\langle Z_7 \rangle$
- `cudaq.estimate_resources`: 69 gates, depth 21: ry 30, cx 11, crz 7, x 14, crx 7

Training and evaluation, line by line (Section 7)

```
readout = cudaq.spin.z(output_qubit) # Z on qubit 7
def predict(theta, images):
    return np.array([cudaq.observe(qcnn, readout,
        list(image), list(theta), kinds, qubit_a,
        qubit_b, layers).expectation() for image in images])

def loss(theta):
    outputs = predict(theta, train_x)
    history["loss"].append(np.mean((outputs - train_y)**2))
    history["accuracy"].append(
        np.mean(np.sign(outputs) == train_y))
    return history["loss"][-1]

theta_initial = rng.uniform(0, INITIAL_PARAMETER_SCALE,
                            n_parameters)
result = minimize(loss, theta_initial, method="COBYLA",
                  options={"maxiter": MAX_ITERATIONS})
test_outputs = predict(result.x, test_x)
```

- `predict`: one `observe` per image; exact expectation on a state-vector simulator (no shots)
- `loss`: MSE over the 20 training images; also records the training accuracy for the plots
- initial angles uniform in $[0, 3.14]$ from the same generator (reproducible)
- SciPy COBYLA: linear models of the loss inside a trust region; `maxiter` = number of loss evaluations
- afterwards: training and test outputs, accuracies, final loss, timings, plots and the report
- nothing is exported except the circuit drawing (`exports/qml/qcnn_circuit.txt`)

The implementation at a glance IN REPOSITORY

dataset	synthetic 2×4 line images	loss	MSE on $\langle Z_7 \rangle$ vs. $y = \pm 1$
features	8 pixel values in $[0, 1]$	optimizer	SciPy COBYLA, 100 evaluations
classes	2 (horizontal $+1$, vertical -1), balanced	gradients	none (derivative-free)
split	20 train / 20 test, seed 42	measurement	`cudaq.observe`, exact $\langle Z \rangle$
encoding	angle, $R_Y(\pi x_i)$, 1 qubit per pixel	backend	`qpp-cpu` (recorded); `nvidia` if a GPU is visible
qubits	8 (15 suggested on a GPU)	multi-GPU	not used by `QML.ipynb`
architecture	QCNN: 3 layers, 11 conv + 7 pool blocks	training time	3.4 s recorded (3.6 s re-executed)
circuit	69 gates, depth 21	memory	$16 \cdot 2^8 = 4$ KB state vector
parameters	12 (4 shared per layer)	outputs	plots, report; circuit drawing `.txt`

Memory monitoring, JSON reports and shot-based training are *not* part of `QML.ipynb` (they exist in the other notebooks).

CUDA-Q execution: CPU, GPU, multi-GPU

target	what it does	memory	status here
`qpp-cpu`	CPU state vector	$16 \cdot 2^N$	recorded and re-executed runs
`nvidia`	one-GPU state vector (cuStateVec)	$16 \cdot 2^N$ (fp64)	selected by `"auto"` if a GPU exists; **not executed**
`nvidia, mgpu`	one state vector across GPUs (MPI)	$16 \cdot 2^N / G$ per GPU	not used by `QML.ipynb`
`nvidia, mqpu`	independent circuits on several GPUs in parallel	per circuit	not used; natural fit for QML
`tensornet-mps`	matrix-product states	$\propto N\chi^2$	selectable by name; not executed
`density-matrix-cpu`	noisy simulation	$16 \cdot 4^N$	used for the noise study (lecture)

Why `mqpu` fits QML: training needs *many small* circuits (one per image, per shift); distributing them over GPUs with `cudaq.observe_async` parallelises the loss. `mgpu` helps only when one circuit is too wide for one GPU ($N \gtrsim 32$).

Honesty

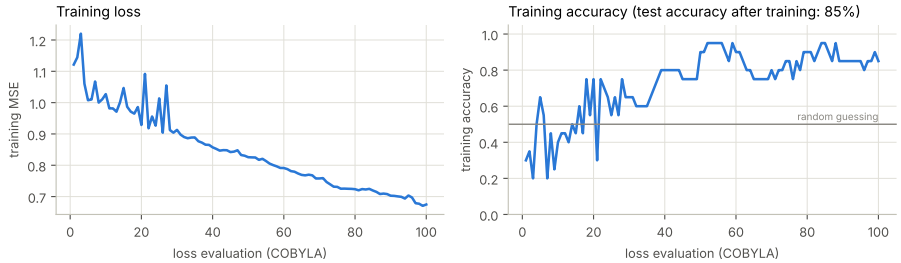
No GPU was available. GPU, multi-GPU and hardware statements are based on the CUDA-Q documentation, not on measurements.

Recorded versus re-executed results

	previously recorded (notebook outputs)	**re-executed for this lecture**
machine	4 vCPU Xeon @ 2.1 GHz, 16.9 GB	4 vCPU cloud container
CUDA-Q / target / GPUs	0.16.0 / `qpp-cpu` / 0	0.16.0 / `qpp-cpu` / 0
qubits, layers, parameters	8, 3, 12	8, 3, 12
gates, depth	69, 21	69, 21
loss evaluations (COBYLA)	100	100
loss: initial $\rightarrow$ final	1.1214 $\rightarrow$ 0.6704	1.1214 $\rightarrow$ 0.6704
training / test accuracy	90 % / 85 %	90 % / 85 %
time per circuit, training time	1.7 ms, 3.4 s	1.7 ms, 3.6 s

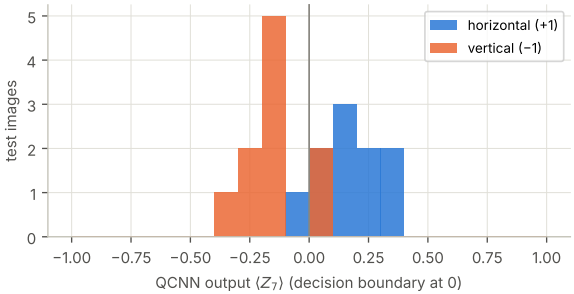
- Same seed, same versions: identical results to all printed digits; timings are machine-specific.
- The re-execution also ran an independent NumPy simulation of the circuit: largest difference in $\langle Z_7 \rangle$ on the test set $6.9e-16$.

Accuracy and loss during training



- each point = one COBYLA loss evaluation (20 circuits); the loss falls from 1.1214 to 0.6704
- training accuracy jumps between neighbouring values because 20 images give steps of 5 %
- the loss is still falling at the end: the 100-evaluation budget, not convergence, stops training

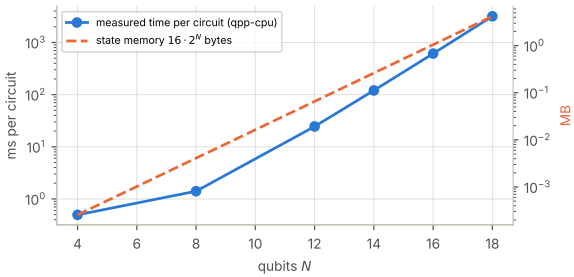
What the trained QCNN outputs



- horizontal images mostly give $\langle Z_7 \rangle > 0$, vertical ones < 0
- but all outputs lie close to 0 (mean $|\langle Z_7 \rangle| = 0.171$): low confidence
- test: TP 9, FN 1, FP 2, TN 8; $F_1 = 0.857$
- test MSE 0.7251 vs. training MSE 0.6704

Performance benchmarking and scalability

ILLUSTRATIVE EXPERIMENT



Time of one QCNN circuit (`cudaq.observe`, `qpp-cpu`, median of 3) vs. qubits.

- 18 qubits: 3179 ms per circuit
- training cost $\approx$ evaluations $\times$ images $\times$ time per circuit
- README (recorded): 15 qubits, $\approx$ 0.35 s per circuit, 60×16 circuits $\approx$ 5.6 min, accuracy only 69 %
- GPU simulation should be much faster (not measured here)

Scalability challenges

Simulation

memory $16 \cdot 2^N$; time $\propto$ gates $\times 2^N$; 30+ qubits need GPUs, 34+ need multiple GPUs or tensor networks.

Training

circuits per epoch $= M\times$(evaluations or shift terms); exact gradients cost $\approx 2\times$(gate occurrences) circuits per image; optimizer steps grow with the number of parameters.

Hardware

shots ($\sim 1/\epsilon^2$), noise (margins shrink), limited connectivity (SWAPs), queue time; data loading per image.

Trainability

barren plateaus, local minima, kernel concentration — all worsen with size.

Quantum advantage — and its limits

What is known

- provable separations exist for artificial problems (e.g. learning based on discrete logarithms, Liu et al. 2021) and for learning from *quantum* data (Huang et al. 2022)
- many claimed exponential speed-ups assume QRAM and low rank; dequantization removes several of them
- no demonstrated practical advantage for classical data such as images

How to test a claim

- strong classical baselines, same data, same budget
- large test sets, error bars, several seeds
- count all costs: encoding, shots, optimizer steps

This repository

The QCNN is simulated on a CPU in seconds; classical methods (rule, kernel SVM) do better on the same task. `QML.ipynb` makes *no* advantage claim, and neither do these slides.

Future research directions

- learning from **quantum data** (sensors, simulations, phases of matter)
- **geometric / equivariant** QML: architectures that respect the symmetries of the data
- trainability without classical simulability: where is the useful middle ground?
- **error mitigation** and fault-tolerant QML
- better **data loading** and kernel design avoiding concentration

Natural extensions of `QML.ipynb`

- GPU / `mqpu` runs at 15–20 qubits
- gradient-based training with per-occurrence shift rules
- shot-based and noisy training
- larger test sets; precision/recall/F1 in the report
- classical baselines inside the notebook

Try it yourself: QML.ipynb

Edit only Section 3:

- NOISE_LEVEL = 0.5: harder task; watch the test accuracy
- MAX_ITERATIONS = 300: does the loss keep falling?
- IMAGE_ROWS, IMAGE_COLS, LINE_LENGTH = 3, 5, 3 with TARGET = "nvidia" on a GPU
- OPTIMIZER = "Nelder-Mead": compare the curves
- different SEEDs: how much does the test accuracy vary?

Reproduce these slides:

- cd presentations/qml_source
- python make_figures.py (re-executes the notebook pipeline and all experiments)
- latexmk -pdf qml_lecture.tex

Test yourself: assessments/QML_Question_Paper.pdf (80 questions) and the separate Answer Book.

Exercises

1. Angle-encode the pixel $x = \frac{2}{3}$ with $R_Y(\pi x)$. Find $P(1)$ and $\langle Z \rangle$.
2. Amplitude-encode $(3, 0, 4, 0)$. How many qubits? Probability that qubit 1 is 1?
3. Compute the notebook's kernel $k(\mathbf{x}, \mathbf{x}')$ for $\mathbf{x} = (0, 1, 0.5)$, $\mathbf{x}' = (0, 0.5, 0.5)$.
4. $f(\theta + \frac{\pi}{2}) = 0.1$, $f(\theta - \frac{\pi}{2}) = 0.7$ for a single $R_Y(\theta)$. Find $f'(\theta)$.
5. How many layers and parameters has the notebook's QCNN for $N = 12$? How many gates?
6. A swap test gives $P(0) = 0.75$. What is $|\langle \psi | \phi \rangle|^2$?
7. How many circuits does one exact gradient of the 8-qubit QCNN loss over 20 images need?
8. A classifier is right on 45 of 50 test images. Give the accuracy with its standard error.

Answers

1. Half-angle $\frac{\pi}{3}$: $P(1) = \sin^2 \frac{\pi}{3} = \frac{3}{4}$, $\langle Z \rangle = \cos \frac{2\pi}{3} = -\frac{1}{2}$.
2. Norm 5; $0.6\,|00\rangle + 0.8\,|10\rangle$; 2 qubits; qubit 1 is always 0: probability 0.
3. $\cos^2 0 \cdot \cos^2 \frac{\pi}{4} \cdot \cos^2 0 = \frac{1}{2}$.
4. $\frac{1}{2}(0.1 - 0.7) = -0.3$.
5. $12 \to 6 \to 3 \to 2 \to 1$: 4 layers, 16 parameters. Conv blocks $(6+5) + (3+2) + (1+1) + 1 = 19$, pool $6 + 3 + 1 + 1 = 11$: $12 + 57 + 44 = 113$ gates.
6. $2(0.75) - 1 = 0.5$.
7. $(4 \cdot 11 + 8 \cdot 7) \times 20 = 2000$.
8. $0.90 \pm \sqrt{0.9 \cdot 0.1/50} = 0.90 \pm 0.042$.

Summary

- ML = data + model + loss + optimizer; judge models on independent test data with error bars and baselines.
- QML models = encoding + parameterized circuit + measured expectation value, trained in a hybrid loop.
- Encodings trade qubits for depth; the fidelity kernel of a product encoding is classically trivial.
- Parameter shift gives exact gradients for Pauli rotations; shared or controlled parameters need care.
- Trainability (barren plateaus), noise, shots and data loading limit QML.
- `QML.ipynb`: 8-qubit QCNN, 12 parameters, 69 gates, 90 % / 85 % — reproduced exactly; simple classical methods do better; no advantage claimed.

Further reading

- M. Schuld, F. Petruccione, *Machine Learning with Quantum Computers* (2021)
- J. Biamonte et al., *Quantum machine learning*, Nature 549 (2017)
- I. Cong, S. Choi, M. Lukin, *Quantum convolutional neural networks*, Nat. Phys. 15 (2019)
- M. Cerezo et al., *Variational quantum algorithms*, Nat. Rev. Phys. 3 (2021)
- NVIDIA CUDA-Q documentation
- This repository: `QML.ipynb`, `assessments/QML_*.pdf`

Syllabus coverage: topics 1–32

#	topic	slides
1	Introduction to machine learning	4
2	Supervised learning	5
3	Unsupervised learning	5
4	Classification and regression	5
5	Training and testing datasets	6
6	Feature engineering	7
7	Loss functions	8
8	Gradient descent	9
9	Neural networks	10, 11
10	Overfitting and underfitting	12
11	Model evaluation	13, 14
12	Classical ML versus QML	15
13	Qubits and quantum states	16
14	Superposition and entanglement	16, 19
15	Quantum gates	20
16	Quantum circuits	21

#	topic	slides
17	Measurement	16, 21
18	Parameterized quantum circuits	22
19	Bloch sphere	17
20	Tensor products	18
21	Unitary operations	20
22	Quantum state preparation	23
23	Introduction to QML	24
24	Quantum data and classical data	24, 47
25	Classical-to-quantum data encoding	25
26	Basis encoding	26
27	Amplitude encoding	26
28	Angle encoding	27
29	Quantum feature maps	28
30	Variational quantum circuits	30
31	Quantum neural networks	31
32	Parameter optimization	34

Syllabus coverage: topics 33–64

#	topic	slides
33	Hybrid quantum-classical learning	32
34	Loss functions and gradients	33, 36
35	Parameter-shift rule	35, 36, 37
36	Measurement and expectation values	21, 33
37	Variational Quantum Classifier	38
38	Quantum Support Vector Machine	39, 40
39	Quantum kernel methods	29, 39, 40
40	Quantum neural networks	31, 41
41	Quantum convolutional neural networks	41, 42, 43
42	Quantum GANs	44
43	Quantum PCA	45
44	Quantum clustering	46
45	Quantum reinforcement learning	46
46	Quantum data classification	47
47	Hybrid quantum-classical architectures	48
48	Barren plateaus	49, 50

#	topic	slides
49	Gradient vanishing	49, 50
50	Expressibility and entanglement	51
51	Circuit depth and trainability	51
52	Noise in QML	52, 53
53	Quantum model evaluation	13, 54, 64
54	Classical baseline comparison	54
55	Dataset preprocessing	55
56	Training and testing pipeline	55, 59
57	CUDA-Q implementation	56, 57, 58, 59, 60
58	GPU-accelerated simulation	61
59	Multi-GPU execution	61
60	Performance benchmarking	65, 62
61	Accuracy and loss visualization	63, 64
62	Scalability challenges	66
63	Quantum advantage and its limitations	67
64	Future research directions	68